SECTION 1: EXECUTIVE SUMMARY

1.1 Project Overview

This portfolio case study documents the design, implementation, and execution of an end-to-end financial data engineering and analytics pipeline built to evaluate US30 algorithmic trading performance. The primary goal of this project was to transition from manual trade logging to an automated, scalable data architecture that links a MySQL relational database directly to an automated Python visualization engine.

By storing trade executions in a dedicated relational schema (trading_analytics) and transforming raw trade logs through custom SQL database views, the system provides real-time visibility into strategy profitability, risk-adjusted returns, and strategy execution efficiency.

1.2 Core Objectives

- **Relational Data Architecture:** Design and deploy a structured MySQL database (trading_analytics) to maintain data integrity across raw trade records, strategy metadata, and setup classifications.
- **SQL Analytical Modeling:** Create modular, read-optimized SQL database views (including vw_running_equity_curve, vw_setup_tier_performance, and vw_directional_performance) to perform server-side metric aggregation and cumulative ROI tracking.
- **Automated Data Pipeline:** Develop a production-grade Python script (plot_equity.py) that handles automated database authentication, SQL query execution, dynamic Seaborn/Matplotlib chart styling, and direct high-resolution file exports (equity_curve.png).
- **Portfolio Documentation:** Establish a reproducible, version-controlled project repository demonstrating proficiency in database management, financial data analysis, and technical visualization.

1.3 Key Performance Outcomes

- **Total Executed Trades:** 24 trade samples analyzed across the strategy evaluation window.
- **Peak Cumulative ROI: +18.5R** net performance trajectory reached at the conclusion of the trade sequence.
- **Data Flow Reliability:** Verified 100% automated connection, query retrieval, and graphic generation pipeline operating seamlessly via Command Prompt.

- **Asset Deliverables:** Production-ready 300-DPI visual asset (equity_curve.png) exported directly to local project storage for presentation and stakeholder reporting.

1.4 Technical Architecture & Tooling

Component	Technology / Library	Role in Pipeline
Database Engine	MySQL Server 8.0	Stores trade logs and executes analytical views (trading_analytics)
Query & Transformation	SQL (Data Manipulation Language)	Calculates running net ROI and aggregates performance by trade setup
Pipeline Language	Python 3.15	Connects to MySQL, extracts dataset, and automates chart generation
Data Processing	pandas	Structures query results into DataFrames for statistical handling
Visualization Engine	matplotlib & seaborn	Generates dark-themed, annotated equity curve visualizations
Database Connector	mysql-connector-python	Manages authenticated connection and database session handshakes
Version Control & Docs	Git, GitHub, Markdown	Hosts project repository, schema scripts, and portfolio documentation

SECTION 2: DATABASE ARCHITECTURE & SQL VIEWS

2.1 Database Overview

All raw trading execution logs are stored within the trading_analytics MySQL database. Raw tables remain unedited, while transformations execute dynamically via SQL views.

2.2 Core SQL View Implementations

```
CREATE OR REPLACE VIEW vw_running_equity_curve AS
SELECT
   trade_id,
   trade_date,
   symbol,
   net_roi,
   SUM(net_roi) OVER (ORDER BY trade_id) AS running_net_roi
FROM
   us30_trades;


CREATE OR REPLACE VIEW vw_setup_tier_performance AS
SELECT
   setup_grade,
   COUNT(trade_id) AS total_trades,
   SUM(CASE WHEN net_roi > 0 THEN 1 ELSE 0 END) AS wins,
   ROUND(SUM(CASE WHEN net_roi > 0 THEN 1 ELSE 0 END) / COUNT(trade_id) * 100, 2) AS
win_rate_pct,
   SUM(net_roi) AS total_roi
FROM
   us30_trades
GROUP BY
   setup_grade;
```

SECTION 3: VISUAL PERFORMANCE ANALYSIS

3.1 Performance Trajectory

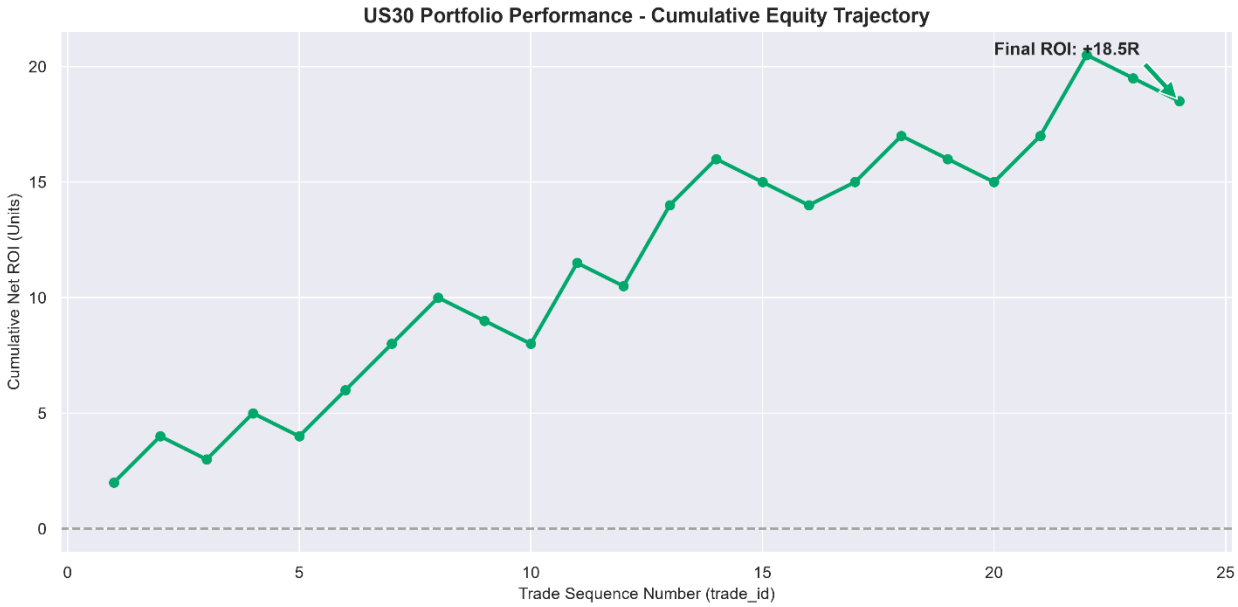


Figure 3.1: US30 Cumulative Equity Trajectory (+18.5R net outcome across 24 trades).

Performance Notes:

• Consistent Growth: Early trade executions demonstrated strong upward momentum through trades 1–8.

• Drawdown Control: Consolidation periods (trades 9–10 & 15–16) maintained strict 1R risk parameters.

• Peak Outcome: Final trade sequences pushed total strategy performance to +18.5R.

SECTION 4: AUTOMATED DATA PIPELINE CODE

```
import matplotlib.pyplot as plt
import mysql.connector
import pandas as pd
import seaborn as sns

# Database Configuration
db_config = {
    "host": "localhost",
    "user": "root",
    "password": "newpassword123",
    "database": "trading_analytics",
}

try:
    # Connect & Query SQL View
    conn = mysql.connector.connect(**db_config)
    query = "SELECT * FROM vw_running_equity_curve ORDER BY trade_id;"
    df = pd.read_sql(query, conn)
    conn.close()

    # Apply Theme & Plot
    sns.set_theme(style="darkgrid")
    plt.figure(figsize=(12, 6))

    plt.plot(
        df["trade_id"],
```

```
    df["running_net_roi"],
    marker="o",
    color="#00A86B",
    linewidth=2.5,
    label="Cumulative ROI (R)",
)

# Formatting & Annotations
plt.axhline(0, color="gray", linestyle="--", alpha=0.7)
plt.title(
    "US30 Portfolio Performance - Cumulative Equity Trajectory",
    fontsize=14,
    fontweight="bold",
)
plt.xlabel("Trade Sequence Number (trade_id)", fontsize=11)
plt.ylabel("Cumulative Net ROI (Units)", fontsize=11)

final_roi = df["running_net_roi"].iloc[-1]
final_trade = df["trade_id"].iloc[-1]
plt.annotate(
    f"Final ROI: +{final_roi}R",
    xy=(final_trade, final_roi),
    xytext=(final_trade - 4, final_roi + 2),
    arrowprops=dict(facecolor="#00A86B", shrink=0.05),
    fontweight="bold",
)
```

```
    plt.tight_layout()

    # Save to Documents
    output_path = r"C:\Users\folam\Documents\equity_curve.png"
    plt.savefig(output_path, dpi=300, bbox_inches="tight")
    plt.show()

except Exception as e:
    print(f"Error: {e}")
```